

SYSTÈMES EMBARQUÉS ET CYBERSÉCURITÉ

Cours

Projet : Gestionnaire de mots de passe embarqué

Bac Pro CIEL - 1ère année

Année 2025-2026

Yahia ISDDIKEN

SOMMAIRE

- 1. Qu'est-ce qu'un système embarqué ?
- 2. Le microcontrôleur Arduino
- 3. Les mémoires dans un système embarqué
- 4. Les données sensibles
- 5. La cybersécurité : principes fondamentaux
- 6. Le chiffrement : protéger les données
- 7. Le chiffrement symétrique
- 8. Le chiffrement XOR en détail
- 9. Exemple pratique de chiffrement XOR
- 10. Limites de sécurité du XOR
- 11. Le clavier T9 : saisie de texte
- 12. Lien avec le métier CIEL

OBJECTIFS DU COURS

Ce cours vous permettra de comprendre les bases des systèmes embarqués et de la cybersécurité. À la fin de ce cours, vous serez capable de :

- ✓ Définir précisément ce qu'est un système embarqué et donner des exemples concrets
- ✓ Expliquer le rôle du microcontrôleur dans un système embarqué
- ✓ Distinguer les 3 types de mémoire (RAM, Flash, EEPROM) et leur utilisation
- ✓ Comprendre ce qu'est une donnée sensible et pourquoi il faut la protéger
- ✓ Expliquer les principes de base de la cybersécurité (CIA)
- ✓ Décrire le fonctionnement du chiffrement symétrique
- ✓ Comprendre et utiliser l'opération logique XOR
- ✓ Chiffrer et déchiffrer un message avec XOR
- ✓ Identifier les failles de sécurité du chiffrement XOR
- ✓ Proposer des améliorations pour sécuriser un système
- ✓ Utiliser un clavier T9 pour saisir du texte
- ✓ Faire le lien avec le métier de technicien CIEL

CHAPITRE 1

QU'EST-CE QU'UN SYSTÈME EMBARQUÉ ?

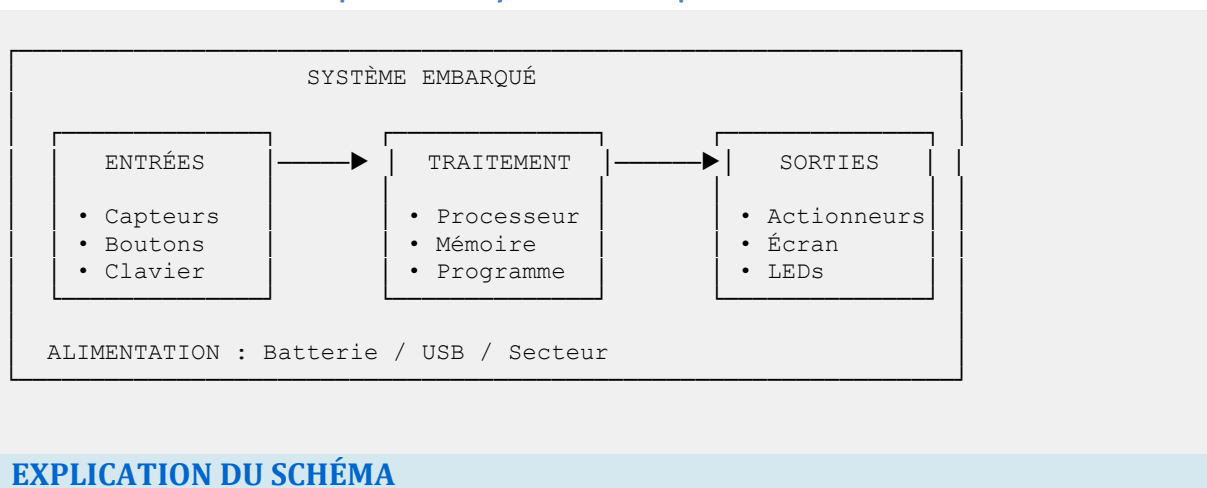
DÉFINITION COMPLÈTE

Un système embarqué est un système informatique autonome, généralement de petite taille, qui est intégré dans un objet ou un appareil pour réaliser une tâche précise et spécifique.

Décomposons cette définition :

- **Système informatique** : Cela signifie qu'il contient un processeur (CPU) qui exécute des instructions, de la mémoire pour stocker des données, et des périphériques pour interagir avec l'extérieur. C'est un véritable ordinateur miniature !
- **Autonome** : Le système embarqué fonctionne tout seul, sans avoir besoin d'être connecté à un ordinateur central. Il a tout ce qu'il faut pour fonctionner : son alimentation, son programme, ses composants.
- **Intégré dans un objet** : Contrairement à un ordinateur classique qui est visible et identifiable, le système embarqué est caché à l'intérieur d'un appareil. Par exemple, dans votre téléphone, dans votre voiture, dans un four micro-ondes, etc.
- **Tâche précise** : Le système embarqué est conçu pour faire UNE chose spécifique, et il la fait bien. Par exemple : contrôler la température d'un four, gérer l'ABS d'une voiture, afficher l'heure sur une montre connectée.

SCHÉMA : Architecture simplifiée d'un système embarqué



EXPLICATION DU SCHÉMA

- Ce schéma montre les 3 composantes principales de TOUT système embarqué :
1. **ENTRÉES** : Comment le système reçoit des informations (capteurs, boutons)
 2. **TRAITEMENT** : Comment il analyse et prend des décisions (processeur + programme)
 3. **SORTIES** : Comment il agit sur le monde réel (LEDs, moteurs, écrans)

CARACTÉRISTIQUES PRINCIPALES

1 Programme unique et dédié

Contrairement à votre ordinateur ou téléphone qui peut exécuter des milliers d'applications différentes, un système embarqué a UN SEUL programme qui ne change jamais (ou rarement). Par exemple, le système embarqué d'un distributeur de billets ne sert qu'à distribuer des billets, rien d'autre.

EXEMPLE CONCRET

L'ordinateur de bord d'une voiture ne peut pas jouer à des jeux vidéo ou naviguer sur Internet. Il est programmé uniquement pour gérer le moteur, l'affichage, la climatisation, etc.

2 Ressources limitées

Les systèmes embarqués ont généralement très peu de mémoire (quelques kilo-octets contre des giga-octets pour un PC) et un processeur peu puissant. Pourquoi ? Car ils n'en ont pas besoin pour leur tâche simple, et cela permet d'économiser de l'énergie et de réduire les coûts.

COMPARAISON DE LA MÉMOIRE RAM :

Arduino Uno :	2 Ko	
Smartphone 2026 :	8 000 000 Ko	

Ratio : Le smartphone a 4 MILLIONS de fois plus de mémoire !
Et pourtant, l'Arduino suffit largement pour notre projet !

EXEMPLE CONCRET

L'Arduino Uno n'a que 2 Ko de RAM, soit 1 million de fois moins qu'un smartphone moderne ! Mais c'est suffisant pour allumer des LEDs, gérer un clavier, et stocker 10 mots de passe.

3 Fonctionnement en temps réel

Beaucoup de systèmes embarqués doivent réagir immédiatement aux événements. Si vous appuyez sur le frein de votre voiture, l'ABS doit réagir en quelques millisecondes, pas dans 5 secondes !

TEMPS DE RÉACTION COMPARÉS :

Humain qui freine :	~250 ms	
ABS de voiture :	~10 ms	
Pacemaker (stimulateur) :	<1 ms	
Airbag :	~5 ms	

Plus le temps de réaction est court, plus c'est critique !

EXEMPLE CRITIQUE

Un pacemaker (stimulateur cardiaque) doit surveiller le cœur en permanence et réagir instantanément si nécessaire. Un retard de quelques secondes pourrait être mortel. C'est pourquoi la fiabilité est ESSENTIELLE.

4 Faible consommation électrique

Les systèmes embarqués sont souvent alimentés par batterie ou doivent fonctionner 24h/24 pendant des années. Ils sont donc conçus pour consommer très peu d'énergie.

CONSOMMATION ÉLECTRIQUE COMPARÉE (en mW - milliwatts) :

Montre connectée :	50 mW	█	(Tient 3-5 jours)
Arduino en veille :	15 mW	█	(Peut tenir des mois)
Smartphone actif :	5000 mW	██████████	(Tient 8-12 heures)
PC portable :	45000 mW	████████████████████	(Tient 3-6 heures)

Plus la consommation est faible, plus l'autonomie est grande !

5 Fiabilité et robustesse

Un système embarqué doit fonctionner sans erreur pendant très longtemps, parfois dans des conditions difficiles (chaleur, froid, vibrations, humidité).

EXEMPLE CONCRET

Le système embarqué d'un avion doit fonctionner parfaitement pendant 20-30 ans, par -50°C en altitude et +40°C au sol, avec des vibrations constantes, sans jamais tomber en panne. C'est pourquoi ces systèmes coûtent très cher !

EXEMPLES DE SYSTÈMES EMBARQUÉS DANS LA VIE QUOTIDIENNE

À RETENIR

Les systèmes embarqués sont PARTOUT autour de vous ! Vous en utilisez des dizaines chaque jour sans même vous en rendre compte.

SYSTÈMES EMBARQUÉS AUTOUR DE VOUS		
MAISON • Lave-linge • Four micro-ondes • Thermostat • Box Internet • Télévision • Réfrigérateur	TRANSPORTS • Voiture (50-100 SE) • Carte bancaire • Badge transport • GPS • Compteur vitesse • Airbags	MÉDICAL • Pacemaker • Pompe insuline • Moniteur • Thermomètre
COMMUNICATION • Smartphone • Tablette • Montre connectée • Oreillettes	INDUSTRIE • Automate (PLC) • Robot industriel • Machine-outil CNC • Capteurs IoT	LOISIRS • Console • Drone • Appareil photo • Imprimante 3D
TOTAL ESTIMÉ : Vous utilisez 20 à 50 systèmes embarqués par jour !		

Détails par catégorie

Dans la maison

- Lave-linge : Contrôle la température, la vitesse d'essorage, le temps de lavage
- Four micro-ondes : Gère la puissance, le temps de cuisson, affiche l'heure
- Thermostat : Régule la température de la maison automatiquement
- Box Internet : Gère la connexion, le WiFi, le routage des données
- Télévision : Contrôle l'image, le son, les menus, les applications

Dans les transports

- Voiture moderne : Contient 50 à 100 systèmes embarqués (ABS, airbags, GPS, climatisation, etc.)
- Carte bancaire sans contact : Crypte et transmet vos données de paiement
- Badge de transport : Stocke votre crédit et valide vos trajets
- Compteur de vitesse : Affiche la vitesse, le kilométrage, la consommation
- GPS : Calcule votre position et votre itinéraire

Dans le médical

- Pacemaker : Surveille et régule le rythme cardiaque
- Pompe à insuline : Injecte automatiquement l'insuline nécessaire
- Moniteur de fréquence cardiaque : Mesure en continu le rythme cardiaque

- Thermomètre électronique : Mesure la température corporelle
- Tensiomètre : Mesure la pression artérielle

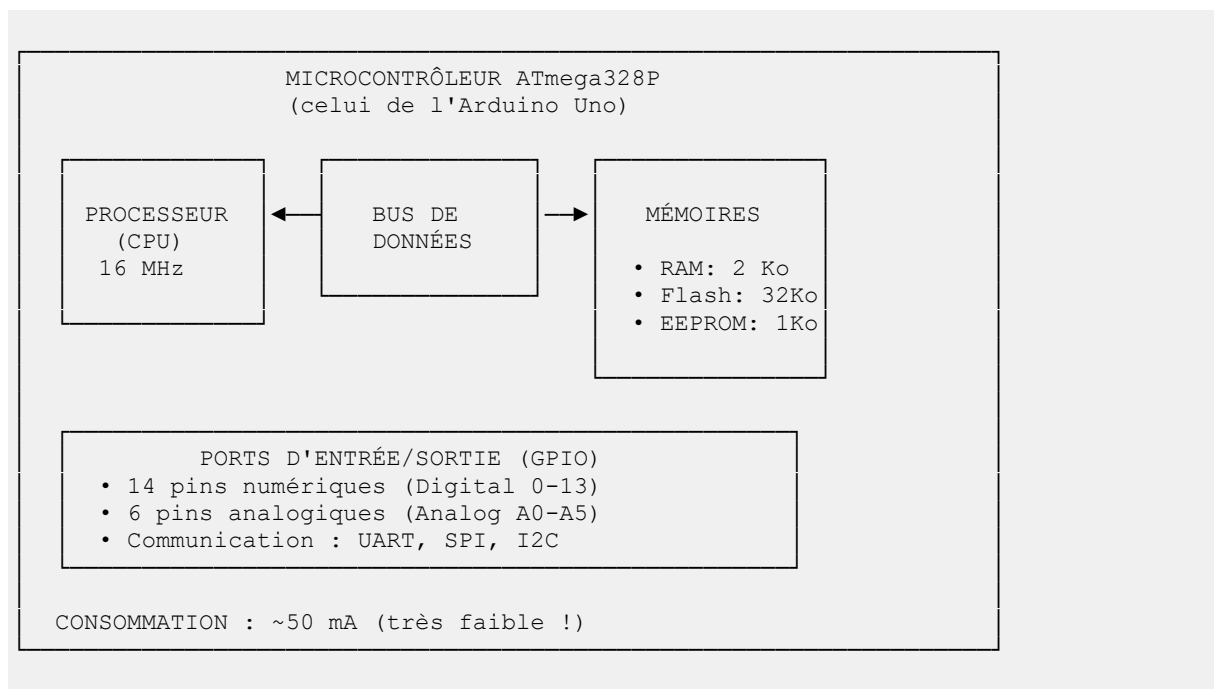
CHAPITRE 2

LE MICROCONTRÔLEUR ARDUINO

QU'EST-CE QU'UN MICROCONTRÔLEUR ?

Un microcontrôleur est un circuit intégré (une puce électronique) qui contient TOUT ce qu'il faut pour faire fonctionner un petit ordinateur : un processeur, de la mémoire, et des ports d'entrée/sortie pour communiquer avec l'extérieur.

SCHÉMA : Anatomie d'un microcontrôleur



Microcontrôleur vs Microprocesseur

MICROCONTRÔLEUR (Arduino)	vs	MICROPROCESSEUR (PC, Smartphone)
TOUT EN UN Une seule puce CPU + RAM + Flash + I/O dans 1 chip Prix : 2-5€ RAM : 2 Ko CPU : 16 MHz Conso : 50 mA Usage : Systèmes embarqués Tâches simples Faible coût		SÉPARÉ Plusieurs puces CPU ▼ RAM (carte) ▼ ROM (disque) Prix : 50-500€ RAM : 8-32 Go CPU : 2-4 GHz Conso : 15-100 W Usage : Ordinateurs Tâches complexes Multitâche

ANALOGIE SIMPLE

Microcontrôleur = Couteau suisse (tout en un, compact, peu cher, fait bien quelques tâches)

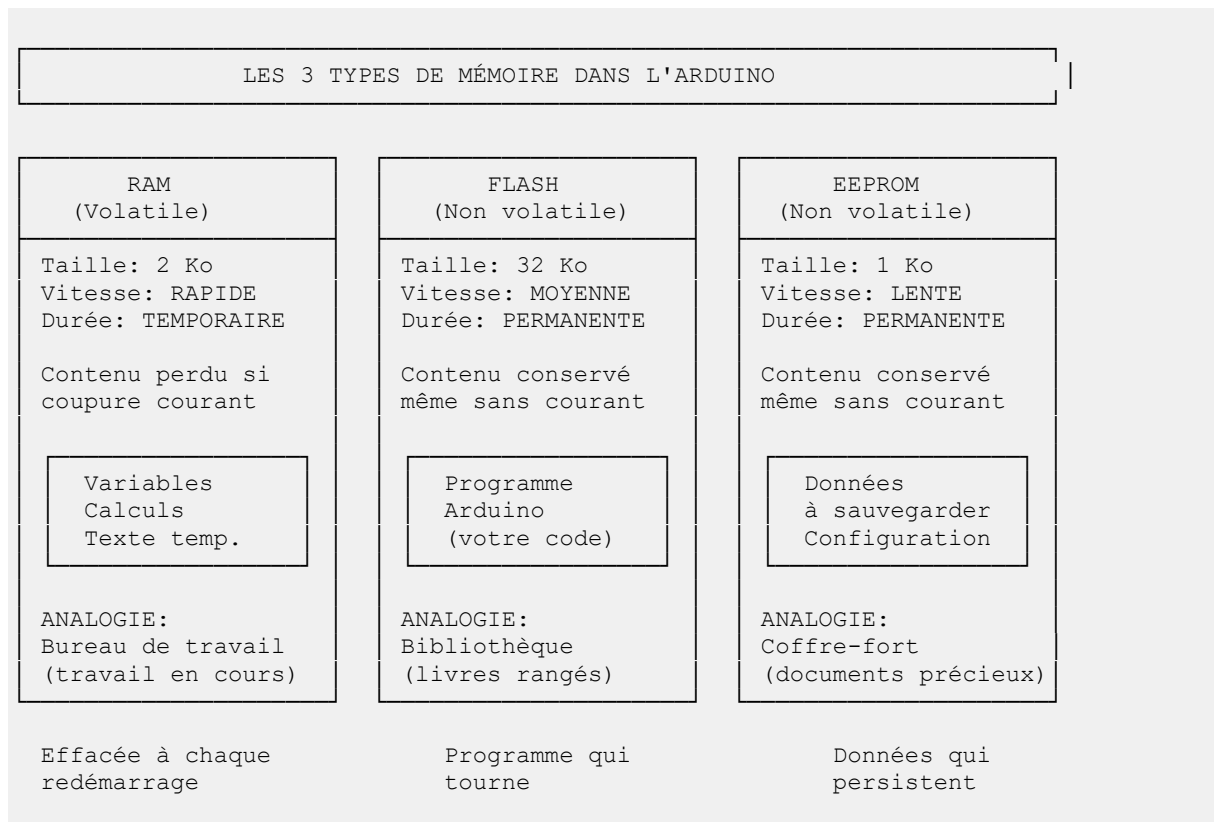
Microprocesseur = Atelier complet (très puissant, modulaire, cher, fait plein de choses)

CHAPITRE 3

LES MÉMOIRES DANS UN SYSTÈME EMBARQUÉ

Un système embarqué a besoin de TROIS types de mémoire différents, chacun ayant un rôle précis. C'est comme dans une maison : vous avez besoin d'un bureau pour travailler (RAM), d'une bibliothèque pour les livres (Flash), et d'un coffre-fort pour les documents importants (EEPROM).

SCHÉMA : Vue d'ensemble des 3 mémoires



1 LA MÉMOIRE RAM (Random Access Memory)

La RAM est la mémoire de travail de l'Arduino. C'est là que le processeur stocke **TEMPORAIREMENT** les données pendant l'exécution du programme. Dès que vous éteignez l'Arduino ou que vous appuyez sur RESET, tout le contenu de la RAM est EFFACÉ.

Caractéristiques de la RAM

MÉMOIRE RAM - DÉTAILS		
Taille sur Arduino Uno : 2048 octets (2 Ko) Vitesse : TRÈS RAPIDE (accès en quelques nanosecondes) Volatilité : VOLATILE (perdue sans alimentation)		
<table border="1"><thead><tr><th>UTILISATION DE LA RAM</th></tr></thead><tbody><tr><td>1. Variables du programme (int, char, String...) 2. Tableaux temporaires 3. Pile d'appels de fonctions 4. Données en cours de traitement</td></tr></tbody></table>	UTILISATION DE LA RAM	1. Variables du programme (int, char, String...) 2. Tableaux temporaires 3. Pile d'appels de fonctions 4. Données en cours de traitement
UTILISATION DE LA RAM		
1. Variables du programme (int, char, String...) 2. Tableaux temporaires 3. Pile d'appels de fonctions 4. Données en cours de traitement		
ATTENTION : Seulement 2 Ko disponibles ! Un String de 100 caractères = 100 octets → Maximum ~20 Strings de 100 caractères en RAM		

EXEMPLE CONCRET

Imaginez que vous écrivez sur un tableau blanc. Vous pouvez écrire, effacer, réécrire très rapidement. Mais si quelqu'un efface le tableau (= coupure de courant), tout disparaît ! C'est exactement ça, la RAM.

Exemple d'utilisation de la RAM

CODE ARDUINO :		EN RAM :	
int temperature = 25;	→	temperature: 25 lettre: 'A' message:"Bonjour"	
char lettre = 'A';	→		
String message = "Bonjour";	→		
void loop() {		temperature: 30 lettre: 'B' message:"Bonjour"	
temperature = 30;	→		← Modifié
lettre = 'B';	→		← Modifié
}			
RESET de l'Arduino	→	(vide)	← Tout effacé !

2 LA MÉMOIRE FLASH (Programme Memory)

La mémoire Flash contient le programme Arduino (votre code compilé). C'est une mémoire **NON VOLATILE** : elle garde son contenu même quand l'Arduino est éteint. C'est pour ça que votre programme démarre automatiquement quand vous rallumez l'Arduino !

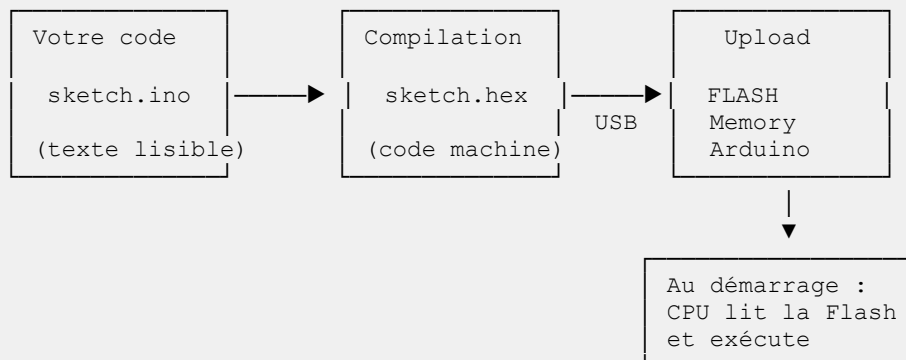
Caractéristiques de la Flash

MÉMOIRE FLASH - DÉTAILS
Taille sur Arduino Uno : 32 768 octets (32 Ko) Vitesse : MOYENNE (plus lente que RAM) Volatilité : NON VOLATILE (garde le contenu) Durée de vie : ~10 000 cycles d'écriture
UTILISATION DE LA FLASH
1. Bootloader (0,5 Ko) - Programme de démarrage 2. Votre programme Arduino (sketch compilé) 3. Constantes définies avec PROGMEM
PROCESSUS : 1. Vous écrivez du code (.ino) 2. Arduino IDE le compile (.hex) 3. Le .hex est transféré dans la Flash via USB 4. Au démarrage, le processeur lit la Flash et exécute

ANALOGIE

La Flash, c'est comme un livre imprimé. Une fois écrit (compilé et uploadé), le contenu ne change plus. Vous pouvez le lire des milliers de fois sans l'abîmer. Mais pour le modifier, il faut ré-imprimer le livre entier (re-compiler et re-upload).

PROCESSUS D'UPLOAD DU PROGRAMME DANS LA FLASH :



3 LA MÉMOIRE EEPROM (Electrically Erasable Programmable ROM)

L'EEPROM est une mémoire NON VOLATILE qui peut être modifiée par le programme. C'est parfait pour sauvegarder des données qui doivent persister entre les redémarrages : paramètres, scores de jeu, dernière valeur d'un capteur, mots de passe, etc.

Caractéristiques de l'EEPROM

MÉMOIRE EEPROM - DÉTAILS
Taille sur Arduino Uno : 1024 octets (1 Ko) Vitesse : LENTE (~3 ms par écriture !) Volatilité : NON VOLATILE (garde le contenu) Durée de vie : ~100 000 cycles d'écriture PAR CELLULE
UTILISATION DE L'EEPROM
1. Sauvegarder des paramètres de configuration 2. Stocker des mots de passe (comme dans notre projet!) 3. Conserver le dernier état d'un appareil 4. Compteurs qui persistent (nombre de démarrages...)
ATTENTION AUX ÉCRITURES TROP FRÉQUENTES ! Après 100 000 écritures, une cellule peut se dégrader → Ne PAS écrire dans une boucle à chaque milliseconde !

RÈGLE D'OR DE L'EEPROM

N'écrivez dans l'EEPROM que quand c'est VRAIMENT nécessaire !

- ✓ BON : Sauvegarder un mot de passe saisi par l'utilisateur (1 fois)
- ✗ MAUVAIS : Sauvegarder la température toutes les 10 ms dans une boucle

Si vous écrivez trop souvent, l'EEPROM va s'user prématurément.

Exemple d'utilisation de l'EEPROM

CODE ARDUINO pour écrire/lire dans l'EEPROM :

```
#include <EEPROM.h>

void setup() {
  // Écrire la valeur 42 à l'adresse 0
  EEPROM.write(0, 42);

  // Écrire un String (plus complexe)
  String motDePasse = "SECRET123";
  for (int i = 0; i < motDePasse.length(); i++) {
    EEPROM.write(10 + i, motDePasse[i]); // À partir de l'adresse 10
  }
}

void loop() {
  // Lire la valeur à l'adresse 0
  int valeur = EEPROM.read(0); // Retourne 42

  // Lire le String
```

```
String motDePasse = "";
for (int i = 0; i < 9; i++) {
  char c = EEPROM.read(10 + i);
  motDePasse += c;
}
// motDePasse contient maintenant "SECRET123"
}
```

MÉMOIRE EEPROM (après exécution) :

Adr	0	1	2	...	10	11	12	13	14
Val	42	0	0	...	'S'	'E'	'C'	'R'	'E'

Ces données restent MÊME SI on éteint l'Arduino !

TABLEAU RÉCAPITULATIF DES 3 MÉMOIRES

CARACTÉRISTIQUE	RAM	FLASH	EEPROM
Taille	2 Ko	32 Ko	1 Ko
Vitesse	TRÈS RAPIDE (nanosec.)	MOYENNE (microsec.)	LENTE (3 ms/écriture)
Volatile ?	OUI Perdu si OFF	NON Garde contenu	NON Garde contenu
Modifiable ?	Constamment	Upload seul	Par programme
Durée de vie	Illimitée	~10 000 cycles	~100 000 cycles
Usage typique	Variables Calculs Temporaire	Programme Code Arduino	Sauvegarde Configuration Mots de passe
Analogie	Bureau travail	Bibliothèque	Coffre-fort

DANS NOTRE PROJET DE GESTIONNAIRE DE MOTS DE PASSE :

RAM → Stocke le mot de passe EN COURS de saisie (temporaire)
Stocke les variables de l'interface (texte affiché...)

FLASH → Contient notre programme Arduino complet
Le code qui gère le clavier, l'affichage, le chiffrement

EEPROM → Stocke les mots de passe de manière PERMANENTE
(Dans ce TP, on n'utilisera PAS l'EEPROM, on reste en RAM)

POUR CE TP

Dans ce TP, nous travaillerons **UNIQUEMENT** avec la RAM. Les mots de passe seront donc perdus à chaque redémarrage. C'est une limitation volontaire pour simplifier le code.

En amélioration future, vous pourrez utiliser l'EEPROM pour sauvegarder les mots de passe de manière permanente !

CHAPITRE 4

LES DONNÉES SENSIBLES

QU'EST-CE QU'UNE DONNÉE SENSIBLE ?

Une donnée sensible est une information qui, si elle tombe entre de mauvaises mains, peut causer des problèmes graves : vol d'identité, perte d'argent, violation de la vie privée, etc.

EXEMPLES DE DONNÉES SENSIBLES	
AUTHENTIFICATION • Mots de passe • Code PIN • Réponses secrètes • Empreintes digitales	FINANCIÈRES • Numéro de carte bancaire • Cryptogramme (CVV) • Coordonnées bancaires (IBAN) • Historique d'achats
PERSONNELLES • Numéro de sécurité sociale • Adresse domicile • Numéro de téléphone • Email • Date de naissance	MÉDICALES • Dossier médical • Analyses médicales • Ordonnances • Groupe sanguin
COMMUNICATIONS • Messages privés • Emails confidentiels • Historique de navigation	PROFESSIONNELLES • Secrets industriels • Brevets • Fichiers clients • Contrats

CONSÉQUENCES D'UNE FUITE DE DONNÉES

Si vos données sensibles sont volées, les conséquences peuvent être :

- Vol d'identité (quelqu'un se fait passer pour vous)
- Perte d'argent (piratage de compte bancaire)
- Chantage (divulgarion de données privées)
- Usurpation de compte (Facebook, Gmail...)
- Atteinte à la réputation

POURQUOI PROTÉGER LES DONNÉES ?

RAISONS DE PROTÉGER LES DONNÉES

1. OBLIGATIONS LÉGALES

Le RGPD (Règlement Général sur la Protection des Données)
impose aux entreprises de protéger les données personnelles
Sanctions : jusqu'à 20 millions d'euros d'amende !

2. ÉTHIQUE ET RESPECT

Les utilisateurs nous confient leurs données
→ Nous devons les respecter et les protéger

3. SÉCURITÉ ÉCONOMIQUE

Une fuite de données = perte de confiance = perte de clients
Coût moyen d'une cyberattaque : 4,2 millions \$ (2023)

4. SÉCURITÉ NATIONALE

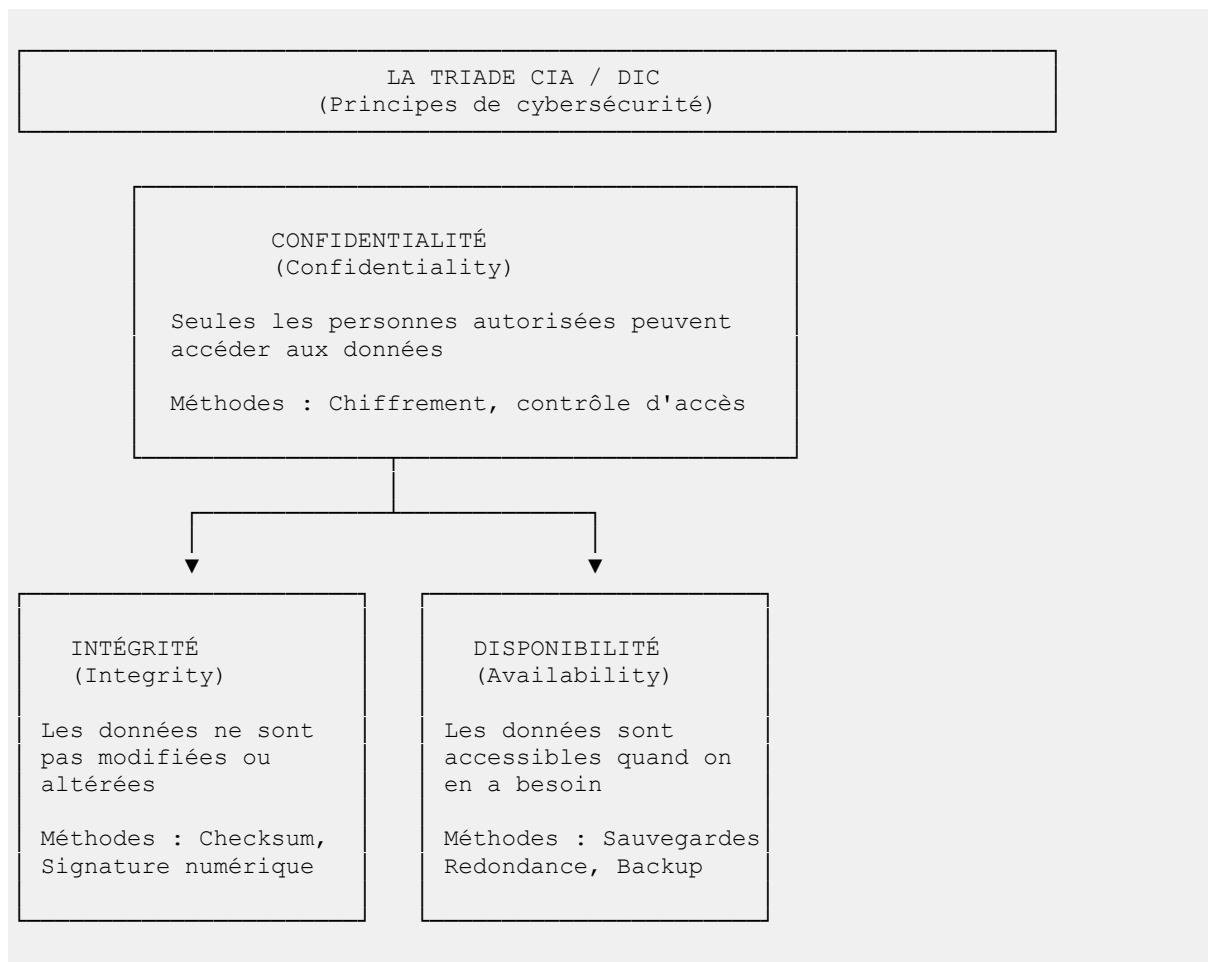
Certaines données (infrastructure, défense) sont critiques
pour la sécurité d'un pays

CHAPITRE 5

LA CYBERSÉCURITÉ : PRINCIPES FONDAMENTAUX

LA TRIADE CIA

En cybersécurité, on parle de la triade CIA (en anglais) ou DIC (en français). Ce sont les 3 piliers fondamentaux de la sécurité informatique :



Détails des 3 piliers

1 CONFIDENTIALITÉ

DÉFINITION

Garantir que seules les personnes **AUTORISÉES** peuvent lire les données.

EXEMPLE CONCRET :

Situation : Vos mots de passe stockés sur l'Arduino

- ✅ **CONFIDENTIEL** : Les mots de passe sont chiffrés (XOR)
→ Un pirate qui vole l'Arduino ne peut pas les lire directement
- ❌ **NON CONFIDENTIEL** : Les mots de passe sont en clair dans la RAM
→ N'importe qui peut les lire avec un simple programme

2 INTÉGRITÉ

DÉFINITION

Garantir que les données n'ont PAS été modifiées ou altérées.

EXEMPLE CONCRET :

Situation : Transfert d'un fichier par Internet

- ✓ INTÈGRE : Le fichier reçu a le même checksum (empreinte) que le fichier envoyé → Aucune modification
- ✗ NON INTÈGRE : Le checksum est différent
→ Le fichier a été modifié (virus ?) ou corrompu

3 DISPONIBILITÉ

DÉFINITION

Garantir que les données et services sont ACCESSIBLES quand on en a besoin.

EXEMPLE CONCRET :

Situation : Site web d'une banque

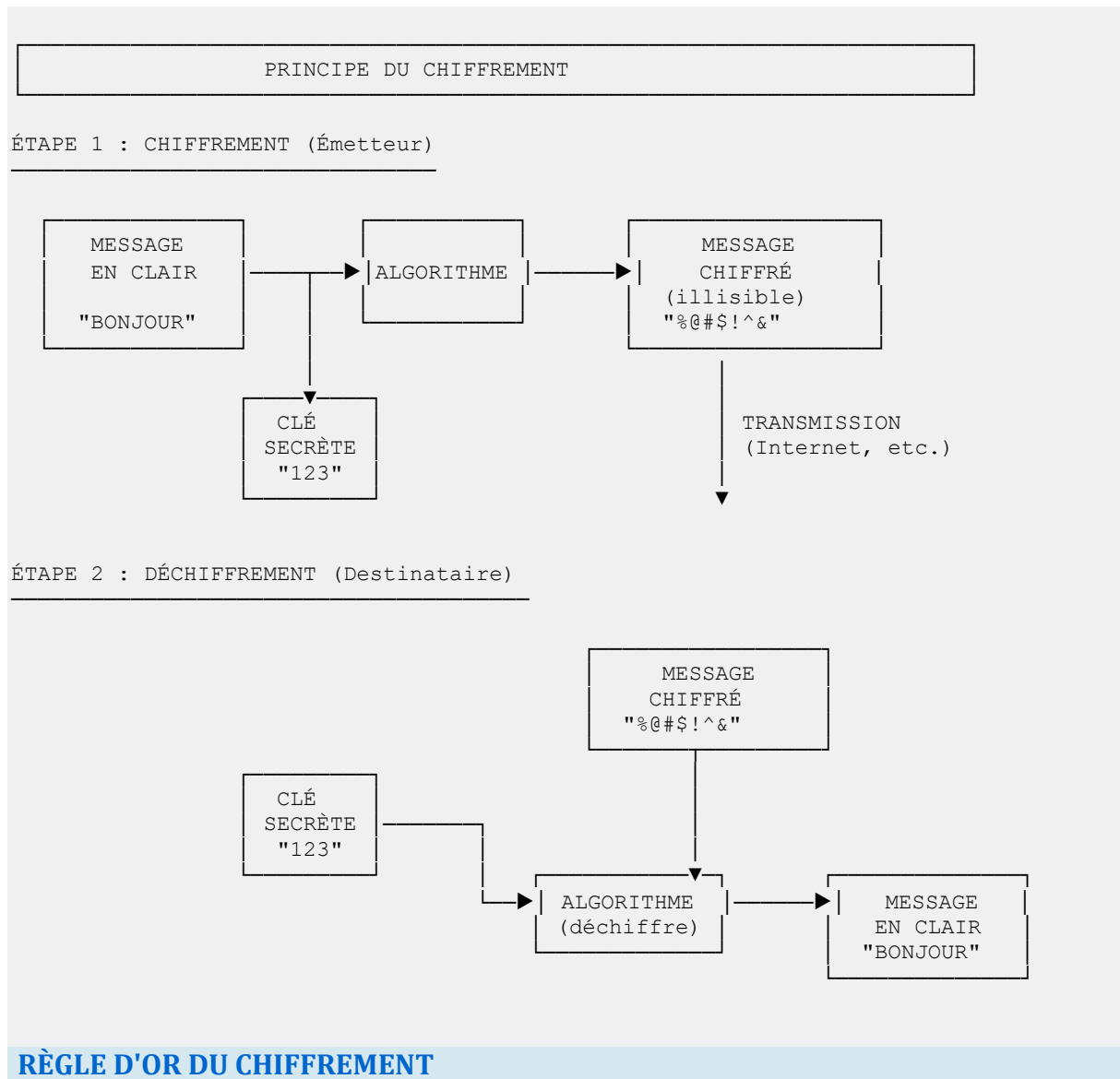
- ✓ DISPONIBLE : Le site fonctionne 24h/24, 7j/7
Même en cas de panne d'un serveur (redondance)
- ✗ NON DISPONIBLE : Attaque DDoS (Déni de Service)
→ Le site est inaccessible pendant des heures
→ Les clients ne peuvent plus accéder à leurs comptes

CHAPITRE 6

LE CHIFFREMENT : PROTÉGER LES DONNÉES

QU'EST-CE QUE LE CHIFFREMENT ?

Le chiffrement est une technique qui permet de rendre un message **ILLISIBLE** pour toute personne qui ne possède pas la clé de déchiffrement. C'est comme écrire en code secret !



Seules les personnes qui possèdent la CLÉ peuvent déchiffrer le message.

Sans la clé, le message chiffré est totalement **ILLISIBLE** et **INUTILISABLE**.

VOCABULAIRE IMPORTANT

VOCABULAIRE DU CHIFFREMENT

TEXTE EN CLAIR (Plaintext)

- Le message original, lisible par tous
- Exemple : "MOT_DE_PASSE"

TEXTE CHIFFRÉ (Ciphertext)

- Le message après chiffrement, illisible
- Exemple : "#@%!^&*()_"

CLÉ (Key)

- L'information secrète nécessaire pour chiffrer/déchiffrer
- Exemple : le nombre 42, ou un mot "SECRET"

ALGORITHME (Algorithm)

- La méthode mathématique utilisée pour chiffrer
- Exemple : XOR, AES, RSA...

CHIFFRER (Encrypt)

- Transformer un texte clair en texte chiffré

DÉCHIFFRER (Decrypt)

- Transformer un texte chiffré en texte clair (avec la clé)

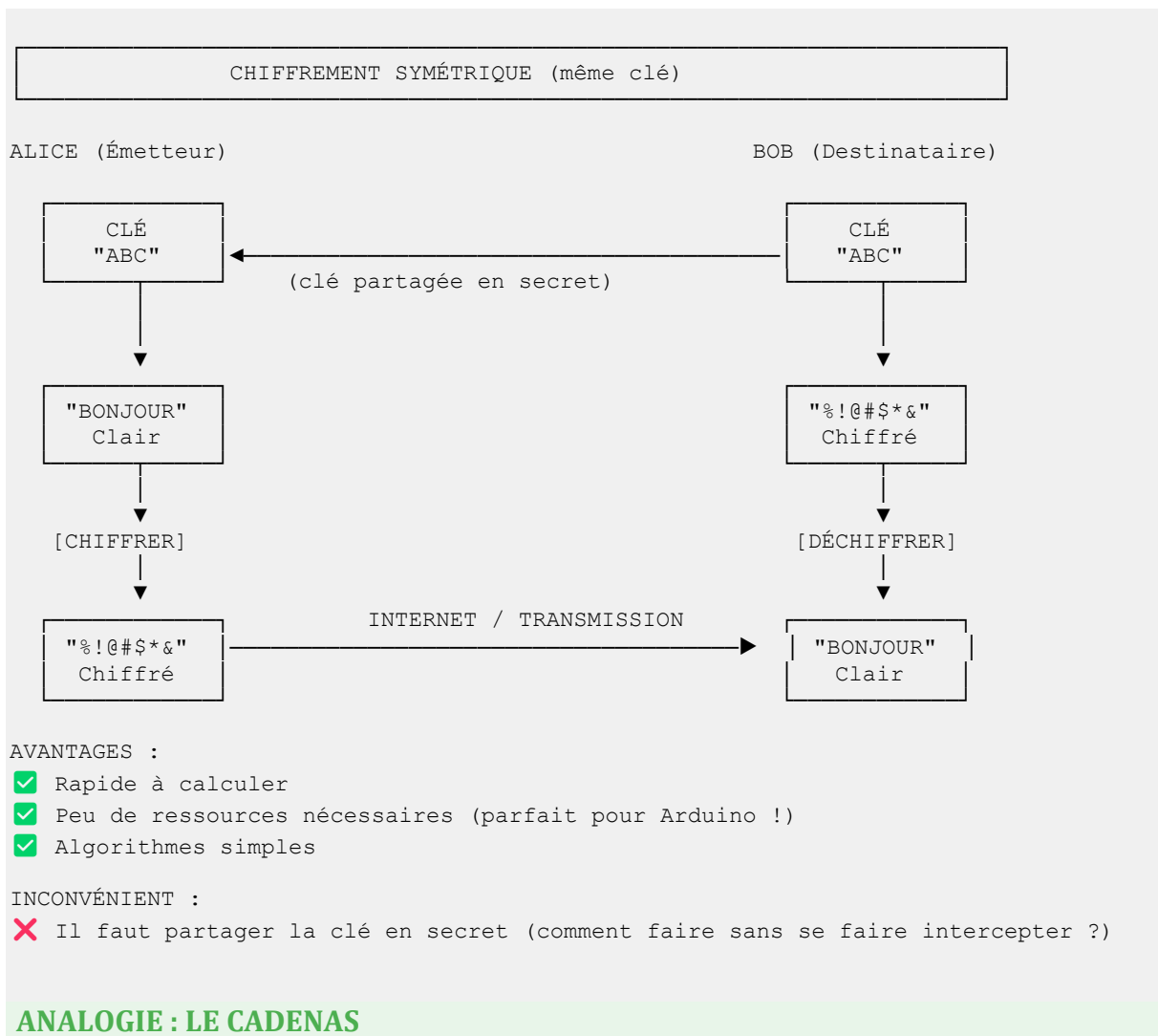
DÉCRYPTER

- Casser un chiffrement SANS avoir la clé (piratage !)
- Ce n'est PAS la même chose que "déchiffrer" !

CHAPITRE 7

LE CHIFFREMENT SYMÉTRIQUE

Le chiffrement symétrique utilise LA MÊME CLÉ pour chiffrer ET déchiffrer. C'est comme une serrure : la même clé ouvre et ferme la porte.



Imaginez un cadenas avec une seule clé qui peut à la fois FERMER et OUVRIR.

Alice ferme le cadenas avec la clé, envoie le cadenas fermé à Bob.

Bob ouvre le cadenas avec LA MÊME clé (qu'Alice lui a donnée avant).

Le problème : Comment donner la clé à Bob sans que Charlie l'intercepte ?

CHAPITRE 8

LE CHIFFREMENT XOR EN DÉTAIL

QU'EST-CE QUE LE XOR ?

XOR signifie "eXclusive OR" (OU exclusif en français). C'est une opération logique qui compare deux bits (0 ou 1) et retourne un résultat selon une règle simple.

TABLE DE VÉRITÉ DU XOR

TABLE DE VÉRITÉ XOR			
BIT A	BIT B	A XOR B	EXPLICATION
0	0	0	Les deux bits sont IDENTIQUES → 0
0	1	1	Les deux bits sont DIFFÉRENTS → 1
1	0	1	Les deux bits sont DIFFÉRENTS → 1
1	1	0	Les deux bits sont IDENTIQUES → 0
RÈGLE SIMPLE :			
Si A et B sont DIFFÉRENTS → Résultat = 1			
Si A et B sont IDENTIQUES → Résultat = 0			
On peut résumer par : XOR détecte les DIFFÉRENCES			

ANALOGIE SIMPLE

Imaginez deux interrupteurs (A et B). Une lampe s'allume SEULEMENT si les deux interrupteurs sont dans des positions DIFFÉRENTES.

- A=bas, B=bas → Lampe éteinte (même position)
- A=bas, B=haut → Lampe allumée (positions différentes)
- A=haut, B=bas → Lampe allumée (positions différentes)
- A=haut, B=haut → Lampe éteinte (même position)

EXEMPLE VISUEL DE XOR

Exemple : Calculer 5 XOR 3

ÉTAPE 1 : Convertir en binaire

5 en décimal = 0 0 0 0 0 1 0 1 (binaire sur 8 bits)
3 en décimal = 0 0 0 0 0 0 1 1 (binaire sur 8 bits)

ÉTAPE 2 : Appliquer XOR bit par bit

Position :	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Nombre 5 :	0	0	0	0	0	1	0	1
Nombre 3 :	0	0	0	0	0	0	1	1
XOR :	0	0	0	0	0	1	1	0

↓

↓

↓

↓

↓

↓

↓

↓

↓

0=0

0=0

0=0

0=0

0=0

1≠0

0≠1

1=1

même

même

même

même

même

DIFF

DIFF

même

↓

↓

↓

↓

↓

↓

↓

↓

↓

0

0

0

0

0

1

1

0

ÉTAPE 3 : Reconvertir en décimal

0 0 0 0 0 1 1 0 (binaire) = 6 (décimal)

RÉSULTAT : 5 XOR 3 = 6

LA PROPRIÉTÉ MAGIQUE DU XOR

PROPRIÉTÉ FONDAMENTALE

Si on applique XOR DEUX FOIS avec la même clé, on retrouve le message original !

Message XOR Clé = Chiffré

Chiffré XOR Clé = Message

C'est pour ça qu'on peut utiliser XOR pour chiffrer ET déchiffrer !

DÉMONSTRATION MATHÉMATIQUE :

Soit :

M = Message original

K = Clé de chiffrement

C = Message chiffré

CHIFFREMENT : $C = M \text{ XOR } K$

DÉCHIFFREMENT : $M = C \text{ XOR } K$

Preuve que ça marche :

```
C XOR K = (M XOR K) XOR K
          = M XOR (K XOR K)      (propriété associative)
          = M XOR 0              (K XOR K = 0 toujours !)
          = M                    (M XOR 0 = M toujours !)
```

Donc on retrouve bien M !

EXEMPLE CONCRET :

Message : 65 (lettre 'A' en ASCII)

Clé : 42

Chiffré : 107 (65 XOR 42 = 107)

Déchiffrement :

Chiffré : 107

Clé : 42

Message : 65 (107 XOR 42 = 65) => On retrouve 'A' !

VISUALISATION BIT PAR BIT :

Message 'A' (65) : 0 1 0 0 0 0 0 1

Clé (42) : 0 0 1 0 1 0 1 0

Chiffré (107) : 0 1 1 0 1 0 1 1

Chiffré (107) : 0 1 1 0 1 0 1 1

Clé (42) : 0 0 1 0 1 0 1 0

Message 'A' (65) : 0 1 0 0 0 0 0 1 => BINGO !

CHAPITRE 9

EXEMPLE PRATIQUE DE CHIFFREMENT XOR

CHIFFRER UN MOT COMPLET AVEC XOR

Voyons maintenant comment chiffrer un mot complet, lettre par lettre, avec XOR.

EXEMPLE COMPLET : CHIFFRER "BONJOUR" AVEC LA CLÉ 42

CLÉ DE CHIFFREMENT : 42 (utilisée pour chaque lettre)

CHIFFREMENT LETTRE PAR LETTRE :

Lettre	Code ASCII	Binaire	Clé (42) Bin	XOR Résultat	Chiffré
B	66	0100 0010	0010 1010	0110 1000	104
O	79	0100 1111	0010 1010	0110 0101	101
N	78	0100 1110	0010 1010	0110 0100	100
J	74	0100 1010	0010 1010	0110 0000	96
O	79	0100 1111	0010 1010	0110 0101	101
U	85	0101 0101	0010 1010	0111 1111	127
R	82	0101 0010	0010 1010	0111 1000	120

RÉSULTAT :

Message original : B O N J O U R
Codes ASCII : 66 79 78 74 79 85 82
Message chiffré : 104 101 100 96 101 127 120

Le message chiffré est ILLISIBLE sans connaître la clé (42) !

DÉCHIFFREMENT (pour vérifier) :

Chiffré	Code chiffré	Binaire	Clé (42) Bin	XOR Résultat	Clair
	104	0110 1000	0010 1010	0100 0010	66 → B
	101	0110 0101	0010 1010	0100 1111	79 → O
	100	0110 0100	0010 1010	0100 1110	78 → N
	96	0110 0000	0010 1010	0100 1010	74 → J
	101	0110 0101	0010 1010	0100 1111	79 → O
	127	0111 1111	0010 1010	0101 0101	85 → U
	120	0111 1000	0010 1010	0101 0010	82 → R

=> On retrouve bien "BONJOUR" !

CODE ARDUINO POUR LE XOR

```
// Fonction pour chiffrer/déchiffrer avec XOR
// (La même fonction fait les deux !)

String xorCipher(String message, int cle) {
    String resultat = ""; // Chaîne vide pour le résultat

    // Pour chaque lettre du message
    for (int i = 0; i < message.length(); i++) {

        char lettre = message[i];           // Récupérer la lettre
        int code = (int)lettre;             // Convertir en code ASCII
        int chiffre = code ^ cle;           // XOR avec la clé (^ = XOR en C++)
        char lettreChiffree = (char)chiffre; // Reconvertir en caractère

        resultat += lettreChiffree;         // Ajouter au résultat
    }

    return resultat; // Retourner le message chiffré/déchiffré
}

// UTILISATION DANS LE PROGRAMME PRINCIPAL :

void setup() {
    Serial.begin(9600);

    String message = "BONJOUR";
    int cle = 42;

    // CHIFFREMENT
    String chiffre = xorCipher(message, cle);
    Serial.print("Message original : ");
    Serial.println(message); // Affiche : BONJOUR
    Serial.print("Message chiffré : ");
    Serial.println(chiffre); // Affiche : heid{x (codes 104,101,100,96,101,127,120)

    // DÉCHIFFREMENT (avec la même fonction !)
    String dechiffre = xorCipher(chiffre, cle);
    Serial.print("Message déchiffré : ");
    Serial.println(dechiffre); // Affiche : BONJOUR
}

void loop() {
    // Rien ici
}
```

EXPLICATIONS :

1. Le symbole ^ en C++ est l'opérateur XOR
Exemple : 65 ^ 42 donne 107
2. La fonction xorCipher() fait à la fois le chiffrement ET le déchiffrement
car XOR est son propre inverse !
3. On convertit les char en int pour faire le XOR, puis on reconvertit en char

CHAPITRE 10

LIMITES DE SÉCURITÉ DU XOR

ATTENTION : XOR N'EST PAS SÉCURISÉ !

Le chiffrement XOR avec une clé fixe (comme dans notre projet) est TRÈS FACILE à casser ! Il est utilisé ici UNIQUEMENT À BUT PÉDAGOGIQUE.

Dans un vrai système de sécurité, on utiliserait AES-256, ChaCha20, ou d'autres algorithmes modernes et robustes.

POURQUOI LE XOR EST FAIBLE ?

FAIBLESSES DU CHIFFREMENT XOR

1 CLÉ TROP COURTE

Dans notre projet, la clé est un simple nombre (42).
Un attaquant peut essayer TOUTES les clés possibles :
0, 1, 2, 3, ..., 255 → Seulement 256 possibilités !

Avec un ordinateur moderne, on peut tester 256 clés en
MOINS D'UNE SECONDE. C'est ce qu'on appelle une
"attaque par force brute".

2 PATTERNS RECONNAISSABLES

Si deux lettres identiques sont chiffrées avec la même clé,
elles donnent le même résultat chiffré.

Exemple : "BONJOUR"
Les deux 'O' donnent le même code chiffré (101).
→ Un attaquant peut détecter ce pattern !

3 PAS D'AUTHENTIFICATION

Rien ne prouve que le message chiffré n'a pas été modifié.
Un attaquant pourrait modifier le message chiffré sans qu'on
s'en rende compte au déchiffrement.

4 CLÉ STOCKÉE EN CLAIR

Dans notre code Arduino, la clé (42) est écrite EN CLAIR
dans le programme. N'importe qui peut lire le code et
trouver la clé !

DÉMONSTRATION : ATTAQUE PAR FORCE BRUTE

Supposons qu'un attaquant intercepte ce message chiffré : [104, 101, 100]
(C'est le début de "BON" chiffré avec la clé 42)

M.ISDDIKEN

L'attaquant va essayer TOUTES les clés possibles :

Clé testée	Déchiffrement de [104, 101, 100]	Résultat lisible ?
0	h e d	✗ Pas de sens
1	i f e	✗ Pas de sens
2	j g f	✗ Pas de sens
...	...	
39	M L I	✗ Pas de sens
40	L M J	✗ Pas de sens
41	N O K	✗ Pas de sens
42	B O N	✓ TROUVÉ ! "BON" a du sens
43	A N M	✗ Pas de sens
...	...	

En quelques millisecondes, l'attaquant a trouvé la clé : 42 !

Temps nécessaire avec un ordinateur moderne : < 0.001 seconde

COMMENT AMÉLIORER LA SÉCURITÉ ?

1 UTILISER UNE CLÉ PLUS LONGUE

Au lieu d'un nombre de 0 à 255, utiliser une longue chaîne

Exemple : "MonMotDePasseSuper Secret123!"

→ Beaucoup plus de combinaisons possibles !

2 UTILISER UN ALGORITHME MODERNE

Remplacer XOR par AES-256, ChaCha20, ou Salsa20

Ces algorithmes sont conçus pour résister aux attaques

3 AJOUTER UN "SEL" (SALT)

Ajouter une valeur aléatoire différente pour chaque mot de passe

Même si deux mots de passe sont identiques, le chiffré sera différent grâce au sel !

4 AJOUTER UN CODE D'AUTHENTIFICATION (HMAC)

Permet de vérifier que le message n'a pas été modifié

C'est une empreinte du message chiffré

5 NE PAS STOCKER LA CLÉ EN CLAIR

Demander à l'utilisateur de saisir un code PIN au démarrage

Utiliser ce PIN pour dériver la clé de chiffrement

→ La clé n'est jamais stockée nulle part !

POUR ALLER PLUS LOIN

Dans un vrai projet professionnel, vous utiliseriez :

- AES-256 pour le chiffrement symétrique
- PBKDF2 ou Argon2 pour dériver une clé depuis un mot de passe
- HMAC-SHA256 pour l'authentification
- Un gestionnaire de clés sécurisé (HSM, TPM...)

Ces sujets seront abordés dans les cours avancés de cybersécurité !

CHAPITRE 11

LE CLAVIER T9 : SAISIE DE TEXTE

QU'EST-CE QUE LE T9 ?

T9 signifie "Text on 9 keys" (Texte sur 9 touches). C'est le système utilisé sur les anciens téléphones mobiles (avant les smartphones) pour écrire du texte avec seulement un clavier numérique.

CLAVIER T9 - DISPOSITION

1	2 ABC	3 DEF	A
4 GHI	5 JKL	6 MNO	B
7 PQRS	8 TUV	9 WXYZ	C
* Effacer	0 Espace	# Valider	D Annuler

MAPPAGE DES TOUCHES :

Touche	Lettres	Nombre d'appuis pour chaque lettre
2	A B C	1×=A 2×=B 3×=C
3	D E F	1×=D 2×=E 3×=F
4	G H I	1×=G 2×=H 3×=I
5	J K L	1×=J 2×=K 3×=L
6	M N O	1×=M 2×=N 3×=O
7	P Q R S	1×=P 2×=Q 3×=R 4×=S
8	T U V	1×=T 2×=U 3×=V
9	W X Y Z	1×=W 2×=X 3×=Y 4×=Z
0	(espace)	1×=espace
*	(effacer)	Supprime le dernier caractère
#	(valider)	Valide la saisie

COMMENT ÉCRIRE AVEC LE T9 ?

PRINCIPE :

1. Appuyer PLUSIEURS FOIS sur une touche pour cycler entre les lettres
2. Attendre 1 seconde OU appuyer sur # pour valider la lettre
3. Continuer avec la lettre suivante

EXEMPLE 1 : Écrire "SALUT"

S : Touche 7 - 4 appuis → P Q R S → On s'arrête sur S → Attendre 1s
A : Touche 2 - 1 appui → A → Attendre 1s
L : Touche 5 - 3 appuis → J K L → On s'arrête sur L → Attendre 1s
U : Touche 8 - 2 appuis → T U → On s'arrête sur U → Attendre 1s
T : Touche 8 - 1 appui → T → Attendre 1s

Séquence complète : 7-7-7-7 [pause] 2 [pause] 5-5-5 [pause] 8-8 [pause] 8 [pause]

EXEMPLE 2 : Écrire "BON"

B : Touche 2 - 2 appuis → A B → On s'arrête sur B
O : Touche 6 - 3 appuis → M N O → On s'arrête sur O
N : Touche 6 - 2 appuis → M N → On s'arrête sur N

Séquence : 2-2 [pause] 6-6-6 [pause] 6-6 [pause]



ATTENTION : LETTRES CONSÉCUTIVES SUR LA MÊME TOUCHE

Pour écrire "AA" (deux A de suite) :



FAUX : 2-2 → Donne B (pas AA !)



VRAI : 2 [pause ou #] 2 [pause ou #] → Donne AA

Il FAUT attendre 1 seconde ou appuyer sur # entre les deux lettres pour que l'Arduino comprenne que c'est une NOUVELLE lettre.

POURQUOI UTILISER LE T9 EN 2026 ?

1. Apprentissage de la programmation avec des contraintes réelles
2. Les systèmes embarqués ont souvent des interfaces limitées
3. Comprendre comment gérer le temps et les événements
4. De nombreux appareils professionnels utilisent encore ce principe :
 - Distributeurs automatiques
 - Terminaux de paiement
 - Systèmes de contrôle industriel
 - Caisses enregistreuses

CHAPITRE 12

LIEN AVEC LE MÉTIER CIEL

COMPÉTENCES DÉVELOPPÉES DANS CE PROJET

COMPÉTENCES BAC PRO CIEL DÉVELOPPÉES DANS CE TP
<u>COMPÉTENCES TECHNIQUES</u> Électronique et câblage : • Lire et comprendre un schéma électrique • Câbler des composants sur breadboard • Calculer une résistance de limitation • Utiliser le bus I2C pour communiquer avec des périphériques • Diagnostiquer une panne matérielle Programmation : • Programmer en C/C++ (langage Arduino) • Gérer des événements temporels (millis()) • Manipuler des chaînes de caractères • Utiliser des bibliothèques externes • Implémenter un algorithme (T9, XOR) • Déboguer un programme avec Serial Monitor Cybersécurité : • Comprendre les principes du chiffrement • Implémenter un algorithme de chiffrement simple • Identifier les vulnérabilités d'un système • Proposer des améliorations de sécurité
<u>COMPÉTENCES PROFESSIONNELLES</u> • Travailler de manière méthodique et rigoureuse • Documenter son travail (schémas, commentaires, rapport) • Tester et valider ses réalisations • Gérer son temps et respecter les délais • Travailler en équipe • Demander de l'aide quand nécessaire

APPLICATIONS PROFESSIONNELLES

CE PROJET VOUS PRÉPARE À DES MÉTIERS RÉELS :

Technicien en systèmes embarqués

- Conception et maintenance de systèmes autonomes
- Industrie, automobile, domotique

Développeur IoT (Internet des Objets)

- Créer des objets connectés
- Capteurs intelligents, domotique, santé connectée

Technicien en cybersécurité

- Sécuriser des systèmes informatiques
- Pentesting, audit de sécurité, SOC

Intégrateur de solutions électroniques

- Assembler et configurer des systèmes complets
- Industrie, télécommunications

Technicien support et maintenance

- Dépanner des équipements électroniques
- Hotline technique, SAV, maintenance

MESSAGE FINAL

Ce projet de gestionnaire de mots de passe embarqué vous permet de développer des compétences essentielles pour votre future carrière dans le domaine CIEL (Cybersécurité, Informatique et réseaux, Électronique).

Même si le chiffrement XOR n'est pas sécurisé en pratique, comprendre son fonctionnement est une excellente base pour aborder des algorithmes plus complexes dans vos études futures.